# Dokumentacja Projektu 1 (SIGK)

**Autorzy:**

- Peng-Yin Chen
- Paweł Niewierowski

---

## 1. Cel projektu

Głównym zadaniem projektowym było zaprojektowanie, zaimplementowanie oraz przeprowadzenie ewaluacji modeli opartych na głębokich sieciach neuronowych (CNN) w celu poprawy jakości obrazów cyfrowych. Projekt skupia się na dwóch kluczowych zagadnieniach modyfikacji obrazu:

1. **Odszumianie (Denoising):** Usuwanie sztucznie wprowadzonego szumu gaussowskiego o różnym natężeniu.
2. **Deblurowanie (Deblurring):** Przywracanie ostrości obrazom poddanym rozmyciu gaussowskiemu.

Implementacja została zrealizowana w języku Python przy użyciu biblioteki PyTorch.

## 2. Metodologia i zbiór danych

Do trenowania i ewaluacji modeli wykorzystano zbiór danych **DIV2K**, powszechnie stosowany w zadaniach rekonstrukcji obrazu.

### 2.1. Przetwarzanie danych

Zgodnie z wymaganiami, obrazy zostały przetworzone do rozdzielczości 256 x 256 pikseli. Proces ten zrealizowano poprzez losowe wycinanie fragmentów (cropping) z oryginalnych obrazów o wysokiej rozdzielczości, co pozwoliło na zwiększenie liczby próbek treningowych i lepszą generalizację modeli. Dane podzielono na zbiór treningowy oraz walidacyjny.

### 2.2. Metryki oceny

Jakość rekonstrukcji obrazu mierzona była za pomocą trzech metryk opisanych w ramach wykładów:

- **PSNR (Peak Signal-to-Noise Ratio):** Mierzy stosunek sygnału do szumu na poziomie pikseli.
- **SSIM (Structural Similarity Index):** Ocenia podobieństwo strukturalne, biorąc pod uwagę luminancję, kontrast i strukturę.
- **LPIPS (Learned Perceptual Image Patch Similarity):** Wykorzystuje cechy głębokie sieci neuronowych do oceny podobieństwa wizualnego zgodnie z ludzką percepcją.

---

## 3. Zadanie 2.2: Odszumianie (Denoising)

### 3.1. Przygotowanie danych i degradacja

W procesie przygotowania danych wykorzystano funkcję **random_noise** z biblioteki **skimage**. Degradacja polegała na dodaniu szumu gaussowskiego do znormalizowanych obrazów (zakres <0.0, 1.0>) dla dwóch wariantów współczynnika sigma 0.01 oraz 0.03.

### 3.2. Metoda bazowa

Wyniki uzyskane przez autorską sieć CNN porównano z klasycznym filtrem dwustronnym (**denoise bilateral**) z biblioteki **skimage**, który jest standardem w tradycyjnym odszumianiu przy zachowaniu krawędzi.

---

## 4. Zadanie 2.3: Deblurowanie (Deblurring)

### 4.1. Przygotowanie danych i degradacja

Rozmycie obrazów zrealizowano za pomocą filtru Gaussa z biblioteki OpenCV (**GaussianBlur**). Zastosowano dwa warianty wielkości kernela: 3 x 3 oraz 5 x 5, co pozwoliło na zróżnicowanie stopnia utraty informacji przestrzennej.

### 4.2. Metoda bazowa

Jako rozwiązanie bazowe do porównania wykorzystano algorytm dekonwolucji **Richardsona-Lucy** (**richardson_lucy** z biblioteki **skimage**), będący iteracyjną metodą przywracania ostrości obrazu.

---

## 5. Architektura modelu i proces treningu

### 5.1. Specyfikacja sieci

Zastosowano lekką architekturę w pełni konwolucyjną, która nie zmienia rozmiaru przestrzennego obrazu. Sieć składa się z czterech warstw splotowych z 64 kanałami pośrednimi i jądrami 3 x 3. Pomiędzy warstwami zastosowano funkcje aktywacji ReLU. W modelu nie zastosowano funkcji sigmoidalnej na wyjściu. Zamiast tego sieć przewiduje poprawkę do obrazu wejściowego, a wynik końcowy powstaje przez dodanie tej poprawki do wejścia. Takie podejście stabilizuje trening i pomaga ograniczyć drobne artefakty. Przed obliczaniem metryk oraz podczas wizualizacji wartości wyjściowe są obcinane do przedziału [0, 1].

### 5.2. Parametry uczenia

Trening przeprowadzono przy użyciu optymalizatora Adam z początkowym learning rate równym 0.001. Dodatkowo zastosowano scheduler StepLR o parametrach step_size = 10 oraz gamma = 0.5, który zmniejsza learning rate w trakcie treningu. Jako funkcję kosztu wykorzystano SmoothL1Loss z parametrem beta = 0.01, co zapewnia kompromis pomiędzy stabilnością uczenia i odpornością na wartości odstające. Modele trenowano przez 25 epok, przy batch size równym 16. Po każdej epoce wykonywano ewaluację na zbiorze walidacyjnym, a zapisywany był model o najlepszej wartości funkcji kosztu na walidacji.

## 6. Wyniki eksperymentów

Wartości w poniższych tabelach zostały wyznaczone na rozłącznym podzbiorze danych walidacyjnych DIV2K. W trakcie treningu do walidacji wykorzystywano 50 pierwszych próbek zbioru walidacyjnego, natomiast końcową ewaluację przeprowadzono na 50 próbkach o indeksach 100-149. Oznacza to, że raportowane wyniki nie pochodzą z tego samego podzbioru, który był używany do wyboru najlepszego modelu podczas treningu.

### Tabela 1: Wyniki dla zadania odszumiania (sigma = 0.01)

| Metoda | PSNR | SSIM | LPIPS |
|---|---|---|---|
| Model CNN (nasz) | 41.3152 | 0.9661 | 0.0130 |
| Denoise Bilateral ($\sigma_r$=0.025, $\sigma_s$=10) | 33.7867 | 0.9339 | 0.1119 |

### Tabela 2: Wyniki dla zadania odszumiania (sigma = 0.03)

| Metoda | PSNR | SSIM | LPIPS |
|---|---|---|---|
| Model CNN (nasz) | 36.9316 | 0.9447 | 0.0444 |
| Denoise Bilateral ( $\sigma_r$=0.075, $\sigma_s$=10) | 26.5887 | 0.8098 | 0.2590 |

### Tabela 3: Wyniki dla zadania deblurowania (kernel 3x3)

| Metoda | PSNR | SSIM | LPIPS |
|---|---|---|---|
| Model CNN (nasz) | 40.2456 | 0.9752 | 0.0062 |
| Richardson-Lucy (num_iter=10) | 23.2268 | 0.8692 | 0.2173 |

### Tabela 4: Wyniki dla zadania deblurowania (kernel 5x5)

| Metoda | PSNR | SSIM | LPIPS |
|---|---|---|---|
| Model CNN (nasz) | 35.7911 | 0.9386 | 0.0426 |
| Richardson-Lucy (num_iter=10) | 20.6839 | 0.7075 | 0.3856 |

# 7. Wizualizacja kilku wybranych próbek

## 7.1. Odszumianie 0.01

Oryginalne | Szum | Odszumione | Odszumione(DB)

## 7.2. Odszumianie 0.03

Oryginalne | Szum | Odszumione | Odszumione(DB)

## 7.3. Deblurowanie 3x3

Oryginalne | Blur | Odblurowane | Odblurowane(RL)

## 7.4. Deblurowanie 5x5

Oryginalne | Blur | Odblurowane | Odblurowane(RL)

# 8. Analiza wizualna i wnioski

## 8.1. Wnioski dla odszumiania

Model CNN wykazuje dużą skuteczność w usuwaniu szumu gaussowskiego. W porównaniu do metody bilateralnej, sieć lepiej radzi sobie z zachowaniem tekstur o niskim kontraście, które filtr bazowy często nadmiernie wygładza ("efekt plasteliny"). Przy wyższym poziomie szumu zauważalne jest lekkie rozmycie detali, jednak ogólna struktura obrazu pozostaje spójna.

## 8.2. Wnioski dla deblurowania

Zadanie deblurowania okazało się trudniejsze pod względem metryk PSNR. Algorytm Richardsona-Lucy, choć skuteczny matematycznie, często wprowadza artefakty typu "ringing" (pierścienie wokół krawędzi). Nasz model CNN generuje obrazy o mniejszej liczbie artefaktów wizualnych, choć przy kernelu 5 x 5 odzyskanie drobnych detali (np. tekstury materiałów) jest ograniczone przez małą liczbę warstw sieci.

## 8.3. Podsumowanie końcowe

Zaproponowana architektura spełniła założenia projektowe, dążąc do wyników lepszych niż rozwiązania bazowe. Wykorzystanie głębokiego uczenia pozwala na bardziej adaptacyjne usuwanie degradacji w porównaniu do sztywnych algorytmów klasycznego przetwarzania obrazów.